

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	K.MAHESH BABU	Reg. No.:	192525122
Section / Batch:	CSA 1214/2025	Date:	11/08/26

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Construct a pipeline scenario containing dependent instructions, branch instructions, and competing resource requirements, and apply techniques such as pipeline stalling, operand forwarding, branch prediction, and resource duplication to minimize performance losses.
2. Develop a computer system model using a hierarchical bus structure to transfer data among the CPU, memory, and I/O devices, and analyse how bus priority, bandwidth, and transfer conflicts affect overall system performance.
3. Develop a processor model that supports multiple instruction issue, out-of-order execution, and speculative execution, and analyse how instruction scheduling, execution units, and simultaneous multithreading affect instruction throughput and processor resource utilization.
4. Design and simulate a processor data path that performs register-to-register data transfers and arithmetic operations such as increment, decrement, addition, and subtraction, and analyse the control signals and execution sequence involved in each operation.
5. Simulate a pipelined processor executing a sequence of arithmetic, memory, and branch instructions, and calculate the pipeline execution time, throughput, and speedup by comparing the results with sequential instruction execution.

Code of Conduct

I K.MAHESH BABU(192525122) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.MAHESH BABU(192525122)

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

Experiment 1: Pipeline Hazards and Performance

Aim

To construct a pipeline scenario containing dependent instructions, branch instructions, and competing resource requirements, and apply stalling, forwarding, branch prediction, and resource duplication.

Scenario

Consider these instructions:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

I3: BEQ R4, R0, LABEL

I4: MUL R6, R7, R8

Here, I2 depends on the result of I1, and I3 depends on the result of I2.

Pipeline stages

IF → ID → EX → MEM → WB

Pipeline without forwarding

Cycle	I1	I2	I3	I4
1	IF			
2	ID	IF		
3	EX	ID		
4	MEM	Stall		
5	WB	Stall	IF	
6		EX	ID	IF
7		MEM	EX	ID
8		WB	MEM	EX
9			WB	MEM
10				WB

Techniques used

1. Operand Forwarding

The result of I1 can be directly forwarded to I2 without waiting for register write-back.

I1 EX —————▶ I2 EX

Forwarding

This reduces unnecessary stalls.

2. Pipeline Stalling

If forwarding cannot solve the dependency, the processor inserts a stall:

Normal: IF → ID → EX

Stall: —

3. Branch Prediction

For BEQ, the processor predicts whether the branch will be taken.

Branch → Prediction → Continue fetching

↓

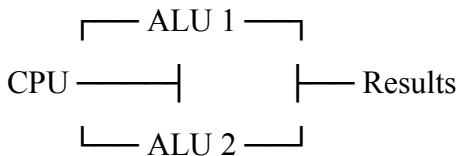
Correct → No major penalty

Wrong → Flush pipeline

4. Resource Duplication

If two instructions require the same hardware resource simultaneously, duplicate the resource where practical.

Example:



Result

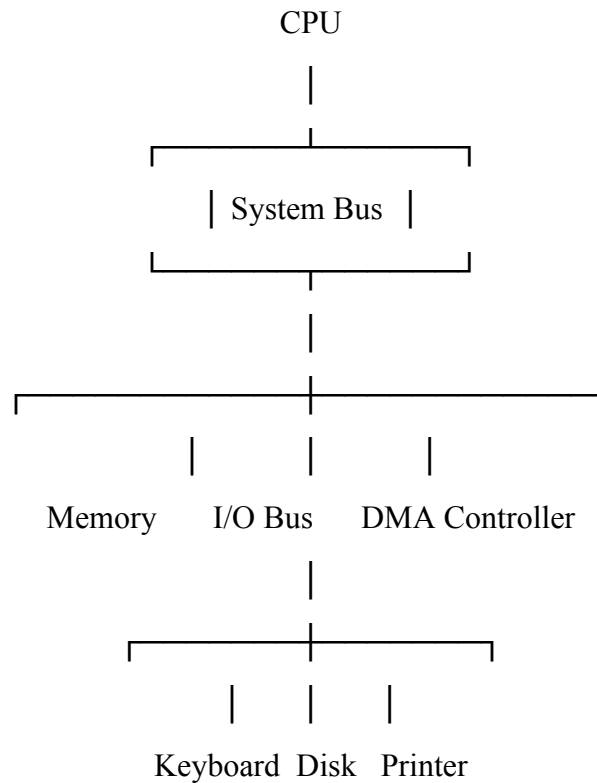
Forwarding reduces data-hazard stalls, branch prediction reduces control-hazard penalties, and resource duplication reduces structural hazards.

Experiment 2: Hierarchical Bus Structure

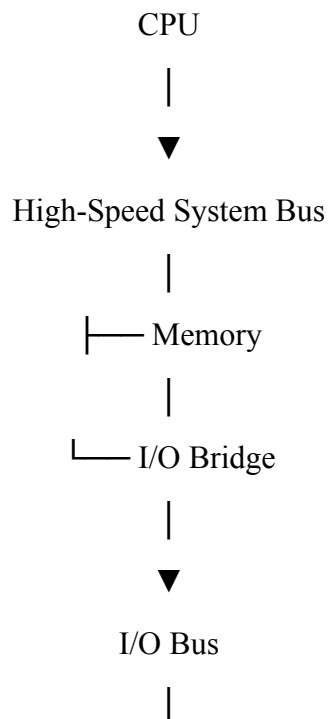
Aim

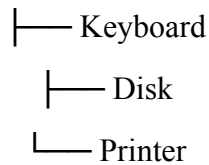
To develop a computer system model using a hierarchical bus structure for transferring data among the CPU, memory, and I/O devices.

Diagram



Bus hierarchy





Bus Priority

When multiple devices request the bus simultaneously, a bus arbitration mechanism determines which device gets access.

Example:

Device Priority

CPU 1

DMA 2

Disk 3

Printer 4

A higher-priority device gets bus access first.

Bandwidth

Bus bandwidth represents the amount of data transferred per unit time.

Bandwidth = Data transferred / Time

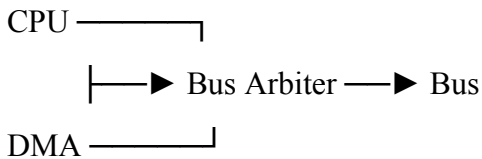
For example, if 100 MB is transferred in 2 seconds:

Bandwidth = 100 / 2

= 50 MB/s

Transfer Conflict

If CPU and DMA request the bus at the same time, arbitration is required.



Result

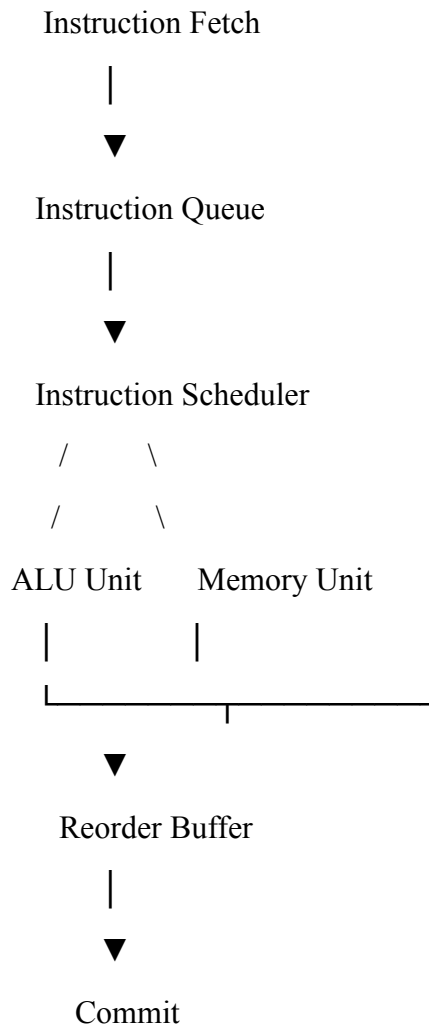
A hierarchical bus separates high-speed CPU/memory communication from slower I/O communication. Proper bus priority and sufficient bandwidth improve overall system performance.

Experiment 3: Multiple Instruction Issue and Out-of-Order Execution

Aim

To develop a processor model supporting multiple instruction issue, out-of-order execution, speculative execution, and simultaneous multithreading.

Basic Model



Multiple Instruction Issue

Instead of executing one instruction per cycle, multiple instructions can be issued.

Example:

Cycle 1 $\rightarrow$ I1 + I2

Cycle 2 $\rightarrow$ I3 + I4

This increases instruction throughput.

Out-of-Order Execution

Suppose:

I1 → waiting for memory

I2 → independent

I3 → independent

The processor can execute:

I2 → Execute

I3 → Execute

I1 → Execute later

instead of waiting for I1.

Speculative Execution

The processor predicts future instructions and executes them before knowing whether they are actually required.

Branch

|



Prediction

|

├── Path A → Execute

└── Path B → possible alternative

If the prediction is incorrect, incorrect speculative instructions are discarded.

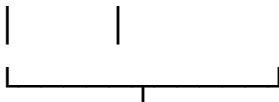
SMT

Simultaneous Multithreading allows multiple threads to use processor resources.

CPU

/ \

Thread 1 Thread 2



Execution Units

Result

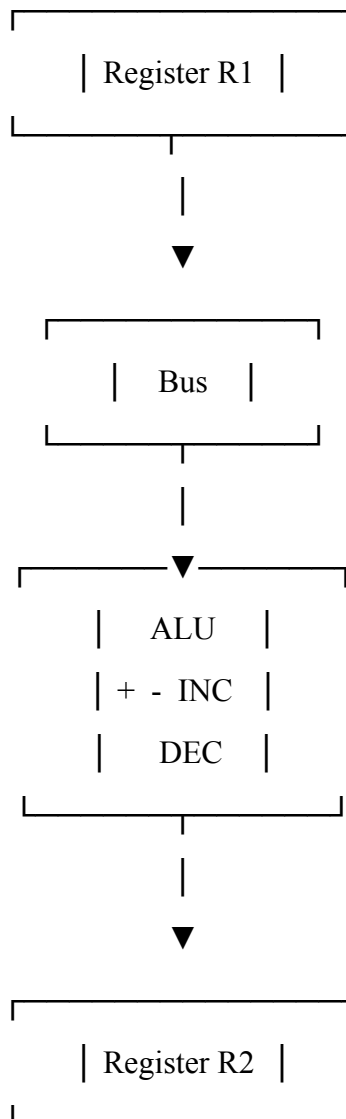
Multiple issue, out-of-order execution, speculative execution, and SMT improve processor utilization and increase instruction throughput.

Experiment 4: Processor Datapath

Aim

To design a processor datapath that performs register-to-register transfers, increment, decrement, addition, and subtraction.

Datapath Diagram



Operations

1. Register Transfer

$R1 \rightarrow R2$

Control signals:

$R1_{out} = 1$

$R2_{in} = 1$

2. Addition

$R3 = R1 + R2$

$R1 \text{ — } \lrcorner$

$\text{└─} \blacktriangleright \text{ALU (+)} \text{ ─} \blacktriangleright R3$

$R2 \text{ — } \lrcorner$

3. Subtraction

$R3 = R1 - R2$

$R1 \text{ — } \lrcorner$

$\text{└─} \blacktriangleright \text{ALU (-)} \text{ ─} \blacktriangleright R3$

$R2 \text{ — } \lrcorner$

4. Increment

$R1 = R1 + 1$

5. Decrement

$R1 = R1 - 1$

Example

Given:

$R1 = 10$

$R2 = 5$

Addition:

$R1 + R2 = 10 + 5 = 15$

Subtraction:

$R1 - R2 = 10 - 5 = 5$

Increment:

$R1 + 1 = 11$

Decrement:

$R1 - 1 = 9$

Result

The datapath successfully performs register transfers and arithmetic operations using registers, buses, ALU, and control signals.

Experiment 5: Pipelined Processor Performance

Aim

To simulate a pipelined processor executing arithmetic, memory, and branch instructions and calculate execution time, throughput, and speedup.

Consider a 5-stage pipeline:

IF → ID → EX → MEM → WB

Suppose there are 5 instructions.

Sequential Execution

Each instruction requires 5 stages.

Total cycles = 5×5

= 25 cycles

Pipelined Execution

For a pipeline with:

- k = 5 stages
- n = 5 instructions

Pipeline cycles:

Cycles = $k + n - 1$

= $5 + 5 - 1$

= 9 cycles

Pipeline Table

Cycle	I1	I2	I3	I4	I5
1	IF				
2	ID	IF			
3	EX	ID	IF		
4	MEM	EX	ID	IF	
5	WB	MEM	EX	ID	IF
6		WB	MEM	EX	ID
7			WB	MEM	EX
8				WB	MEM
9					WB

Speedup = Sequential time / Pipelined time

$$= 25 / 9$$

$$\approx 2.78$$

Therefore, the pipeline provides approximately:

Speedup ≈ 2.78 times

Throughput

After the pipeline is filled, approximately one instruction completes per clock cycle.

Throughput = Instructions / Cycles

$$= 5 / 9$$

$$\approx 0.556 \text{ instructions/cycle}$$

Comparison:

Parameter	Sequential	Pipelined
Instructions	5	5
Stages	5	5
Cycles	25	9
Speedup	1×	2.78×
Completion	One after another	Overlapped

Result

Pipelining reduces the total execution cycles from 25 to 9 for the five-instruction example, giving an idealized speedup of approximately 2.78×